

Nahamcon 2020 Crypto - Oo la la

deuterium — 2020-06-14

Canonical HTML: <https://deut-erium.github.io/WriteUps/2020/nahamconCTF/crypto/Ooo-la-la/2020-06-14-Nahamcon-2020-Ooo-la-la.html>

Abstract

CTF writeup: RSA's primes are a little too neighborly in NahamCon 2020's Ooo-la-la. Alpertron factors the modulus so decryption can proceed.

Ooo-la-la

Challenge

362 Solves

×

Ooo-la-la

100

Uwu, wow! Those numbers are fine!

Download the file below.

 prompt.txt

Flag

Submit

We are provided prompt.txt, the contents of which read

```
N =
33496832406833037520401001871232450767758028386681253257853183150043987785865388662101
98083573169673444543518654385038484177110828274648967185831623610409867689938609495858
551308025785883804091
e = 65537
c =
```

87760575554266991015431110922576261532159376718765701749513766666239189012106797683148
33477144680102104707800312181671082503389480574311258094239998596150968553430987962120
5633997976721084983

Seems like usual RSA, lets see if we can factor N over [alpetron](#)

Electronics Mathematics Programs Contact ESP

Integer factorization calculator

Alpertron > Programs > Integer factorization calculator

Value

871232450767758028386681253257853183150043987785865388662101980835731696734445435186543850384841771108282746489671858316236104098676899386094958585513080257858838040

< >

One numerical expression or loop per line. Example: x=3;x=n(x);c<=100;x-1

Only evaluate Factor Help Config Wizard

Press the **Help** button to get help about this application. Press it again to return to the factorization. Keyboard users can press CTRL+ENTER to start factorization. This is the WebAssembly version.

- 3 349683 240683 303752 040100 187123 245076 775802 838668 125325 785318 315004 398778 586538 866210 198083 573169 673444 543518 654385 038484 177110 828274 648967 185831 623610 409867 689938 609495 858551 308025 785883 804091 (193 digits) = 1 830213 987675 567884 451892 843232 991595 746198 390911 664175 679946 063194 531096 037459 873211 879206 428207 (97 digits) × 1 830213 987675 567884 451892 843232 991595 746198 390911 664175 679946 063194 531096 037459 873211 879206 428213 (97 digits)

Number of divisors: 4

Sum of divisors: 3 349683 240683 303752 040100 187123 245076 775802 838668 125325 785318 315004 398778 586538 866210 198083 573173 333872 518869 790153 942269 863576 811466 141363 967654 951961 769759 816327 671687 933471 054449 544296 660512 (193 digits)

Euler's totient: 3 349683 240683 303752 040100 187123 245076 775802 838668 125325 785318 315004 398778 586538 866210 198083 573166 013016 568167 518616 134698 490644 845083 156570 404008 295259 049975 563549 547303 783631 561602 027470 947672 (193 digits)

Which almost instantly factors the N using [Elliptic Curve Method](#)

Knowing the factors or phi, rest of the job is easy

```
from gmpy2 import invert
p =
18302139876755678844518928432329915957461983909116641756799460631945310960374598732118
79206428207
q =
18302139876755678844518928432329915957461983909116641756799460631945310960374598732118
79206428213

N =
33496832406833037520401001871232450767758028386681253257853183150043987785865388662101
98083573169673444543518654385038484177110828274648967185831623610409867689938609495858
551308025785883804091
e = 65537
c =
87760575554266991015431110922576261532159376718765701749513766666239189012106797683148
33477144680102104707800312181671082503389480574311258094239998596150968553430987962120
5633997976721084983

d = invert(e, (p-1)*(q-1))

pt = pow(c,d,N)
print(bytes.fromhex(hex(pt)[2:]).decode())
```

flag—ooo`la`la`those`are`sexy`primes”

Interactive controls are omitted from this PDF. See the canonical HTML version.